

ASP.NET Core

Introduction

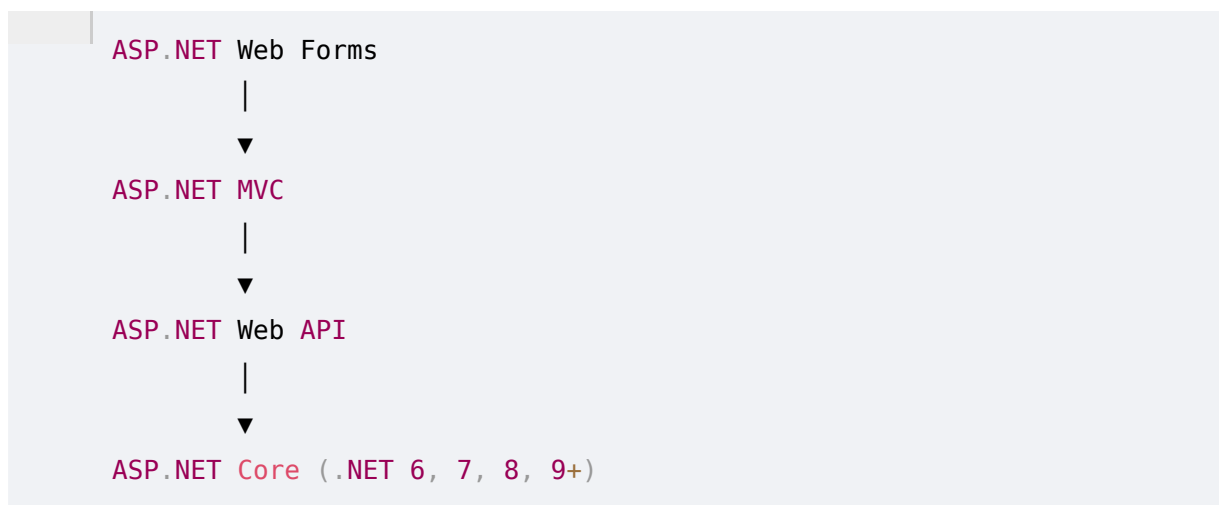
ASP.NET Core is Microsoft's modern, open-source, cross-platform web framework for building web applications, RESTful APIs, microservices, cloud-native applications, and real-time services. It is the successor to the traditional ASP.NET Framework and has been designed from the ground up to provide high performance, modularity, scalability, and flexibility.

ASP.NET Core runs on **Windows**, **Linux**, and **macOS**, making it suitable for cloud deployments, Docker containers, Kubernetes clusters, and on-premises hosting.

Since its release, ASP.NET Core has become the standard framework for enterprise-grade web development using .NET. It powers applications ranging from small internal tools to large-scale distributed systems serving millions of users.

Evolution of ASP.NET

The ASP.NET ecosystem has evolved significantly over the years.



Major improvements introduced with ASP.NET Core include:

- Cross-platform support
- Built-in Dependency Injection
- High-performance request pipeline
- Modular middleware architecture
- Unified MVC and Web API framework
- Cloud-native capabilities

- Minimal APIs
 - Improved security
 - Better testing support
-

Why ASP.NET Core?

Modern enterprise applications require:

- High performance
- Scalability
- Security
- Maintainability
- Cross-platform deployment
- Cloud compatibility

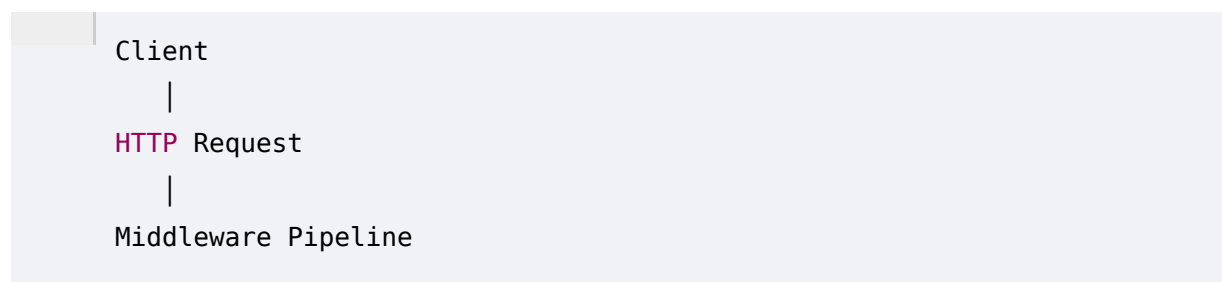
ASP.NET Core addresses these requirements by providing a lightweight, modular architecture that allows developers to include only the components they need.

Key benefits include:

- Open source
 - Cross-platform
 - High throughput
 - Built-in Dependency Injection
 - Excellent tooling
 - Native cloud support
 - REST API development
 - Authentication and authorization
 - Middleware pipeline
 - Easy deployment
-

ASP.NET Core Architecture

A typical ASP.NET Core application follows a layered architecture.



```
|
Routing
|
Controller / Minimal API
|
Business Service
|
Repository
|
Entity Framework Core
|
SQL Server
```

Each layer has a specific responsibility, promoting separation of concerns and easier maintenance.

Project Structure

A standard ASP.NET Core Web API project contains the following folders.

```
Project
|
├─ Controllers
├─ Models
├─ Services
├─ Repositories
├─ Data
├─ Middleware
├─ Configuration
├─ DTOs
├─ appsettings.json
├─ Program.cs
└─ Properties
```

Larger applications may also include folders for:

- Interfaces
- Validators
- Helpers
- Extensions
- Filters
- Background Services

Program.cs

Starting with .NET 6, configuration is centralized in **Program.cs**.

Example:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();

app.UseSwaggerUI();

app.UseHttpsRedirection();

app.UseAuthorization();
```

This file configures services, middleware, routing, authentication, and application startup.

Middleware Pipeline

Middleware components process every HTTP request.

```
Incoming Request
|
HTTPS Redirection
|
Authentication
|
Authorization
|
Routing
|
Controller
```

Response

Each middleware can:

- Process the request
- Modify the request
- Call the next middleware
- Stop the pipeline
- Modify the response

Built-in Middleware

Common middleware components include:

```
app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();

app.UseAuthorization();

app.UseExceptionHandler();

app.UseCors();

app.UseResponseCompression();
```

The order of middleware registration is important because requests pass through the pipeline sequentially.

Controllers

Controllers handle incoming HTTP requests.

Example:

```
[ApiController]
[Route("api/[controller]")]
```

```
public class ProductController : ControllerBase
{
    [HttpGet]
    public IActionResult GetProducts()
    {
        return Ok();
    }
}
```

Controllers provide:

- Routing
 - Model binding
 - Validation
 - HTTP response generation
-

Routing

Routing maps incoming URLs to endpoints.

Attribute routing example:

```
[Route("api/products")]
public class ProductController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok();
    }

    [HttpGet("{id}")]
    public IActionResult GetById(int id)
    {
        return Ok(id);
    }
}
```

Request examples:

```
GET /api/products
```

GET /api/products/5

Model Binding

ASP.NET Core automatically binds incoming request data.

Sources include:

- Route parameters
- Query parameters
- HTTP headers
- Form data
- JSON request body

Example:

```
[HttpPost]
public IActionResult Create(Product product)
{
    return Ok(product);
}
```

The framework converts incoming JSON into a Product object automatically.

Dependency Injection

ASP.NET Core has built-in Dependency Injection.

Register services:

```
builder.Services.AddScoped<IProductService, ProductService>();
```

Inject services:

```
public class ProductController : ControllerBase
{
    private readonly IProductService _service;

    public ProductController(IProductService service)
    {
        _service = service;
    }
}
```

Benefits include:

- Loose coupling
 - Easier testing
 - Better maintainability
-

Configuration

Application settings are stored in **appsettings.json**.

Example:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=.;Database=StoreDB;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information"
    }
  }
}
```

Read configuration:

```
public class ProductService
{
    public ProductService(IConfiguration configuration)
    {
        var connection =
            configuration.GetConnectionString("DefaultConnection");
    }
}
```

Logging

ASP.NET Core includes a built-in logging framework.

Inject logger:

```
public class ProductController : ControllerBase
{
    private readonly ILogger<ProductController> _logger;
```



```
public ProductController(  
    ILogger<ProductController> logger)  
{  
    _logger = logger;  
}  
}
```

Example:

```
_logger.LogInformation("Request received");
```

Logging providers include:

- Console
- Debug
- Event Viewer
- Azure Monitor
- Serilog
- NLog
- Log4Net

Entity Framework Core Integration

Database access is commonly implemented using Entity Framework Core.

Register DbContext:

```
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlServer(  
  
        builder.Configuration.GetConnectionString("DefaultConnection")));
```

Inject DbContext:

```
public class ProductRepository  
{  
    private readonly AppDbContext _db;  
  
    public ProductRepository(AppDbContext db)  
    {  
        _db = db;  
    }  
}
```

```
}  
}
```

Authentication

Authentication verifies the identity of a user.

Common authentication mechanisms include:

- JWT Bearer Tokens
- Cookies
- OAuth 2.0
- OpenID Connect
- Microsoft Identity
- Azure Active Directory

Example:

```
builder.Services.AddAuthentication();  
  
app.UseAuthentication();
```

Authorization

Authorization determines what an authenticated user can access.

Example:

```
[Authorize]  
public class ProductController : ControllerBase  
{  
}  
}
```

Role-based authorization:

```
[Authorize(Roles = "Admin")]
```

Policy-based authorization is also supported for more advanced scenarios.

Exception Handling

Global exception handling improves reliability.

Example:

```
app.UseExceptionHandler("/error");
```

Custom middleware can also be created for:

- Logging exceptions
 - Returning standardized error responses
 - Correlation IDs
-

Swagger

Swagger generates interactive API documentation.

Register services:

```
builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();
```

Middleware:

```
app.UseSwagger();

app.UseSwaggerUI();
```

Access Swagger UI:

```
https://localhost:5001/swagger
```

Swagger allows developers to:

- Explore endpoints
 - Test APIs
 - View request and response schemas
 - Download OpenAPI specifications
-

Filters

Filters provide reusable logic around controller actions.

Types of filters:

- Authorization Filters
- Resource Filters
- Action Filters

- Exception Filters
- Result Filters

Example:

```
public class LoggingFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
    }
}
```

Background Services

Long-running tasks can be implemented using hosted services.

Example:

```
public class WorkerService : BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
    }
}
```

Register:

```
builder.Services.AddHostedService<WorkerService>();
```

Use cases include:

- Scheduled jobs
- Email processing
- Queue consumers
- Data synchronization

CORS

Cross-Origin Resource Sharing (CORS) allows browsers to access APIs from different domains.

Register:

```
builder.Services.AddCors();
```

Middleware:

```
app.UseCors();
```

Typical scenarios:

- Angular frontend calling ASP.NET Core API
- React frontend consuming backend services
- Mobile applications accessing APIs

ASP.NET Core Request Lifecycle

```
graph TD; A[HTTP Request] --> B[Kestrel Web Server]; B --> C[Middleware Pipeline]; C --> D[Routing]; D --> E[Authentication]; E --> F[Authorization]; F --> G[Controller Action]; G --> H[Business Service]; H --> I[Repository];
```

Understanding the request lifecycle is essential for debugging and optimizing applications.

Advantages of ASP.NET Core

- High performance
 - Cross-platform compatibility
 - Open source
 - Built-in Dependency Injection
 - Modular architecture
 - Middleware-based request pipeline
 - Cloud-ready
 - Microservices support
 - REST API development
 - Excellent testing support
 - Native support for Docker and Kubernetes
 - Rich ecosystem and tooling
-

Best Practices

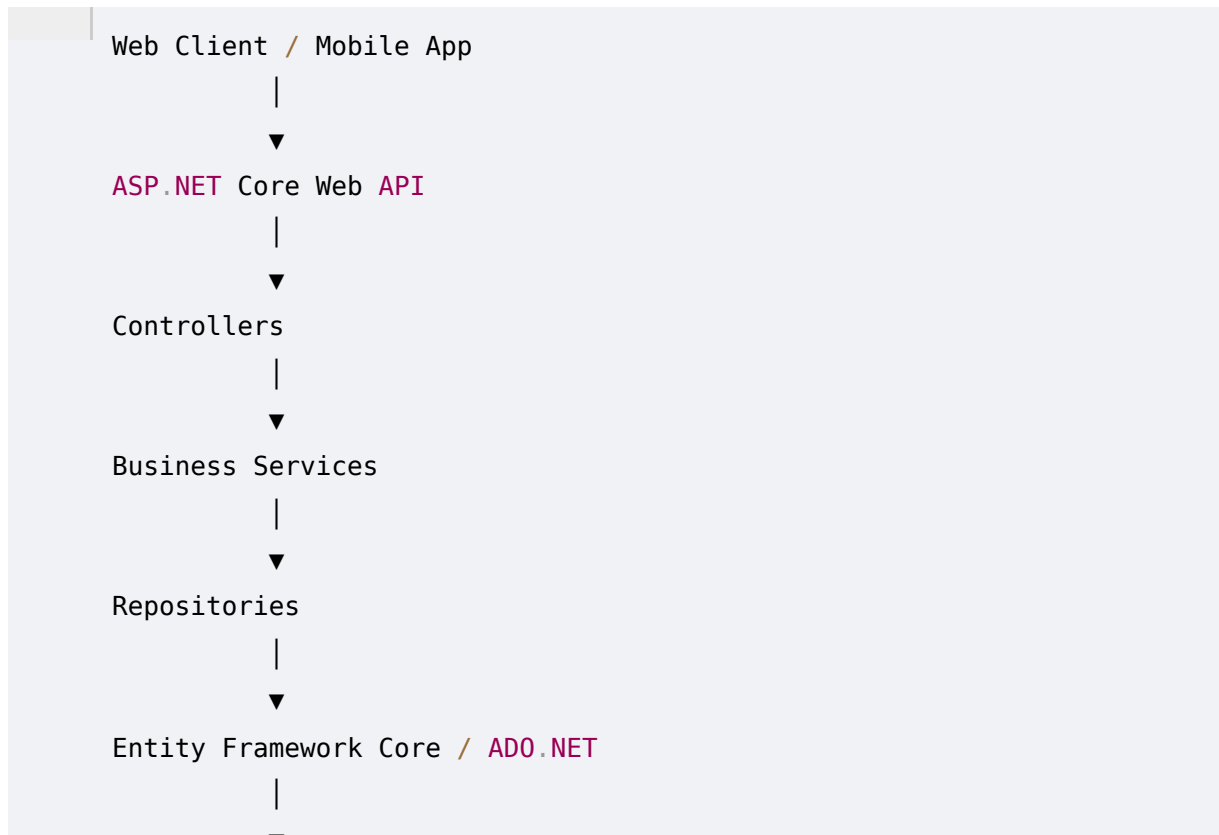
- Follow SOLID principles.
 - Keep controllers thin and move business logic to services.
 - Use Dependency Injection for all external dependencies.
 - Validate incoming models.
 - Use asynchronous programming (`async` / `await`) for I/O operations.
 - Centralize exception handling.
 - Protect sensitive endpoints with authentication and authorization.
 - Use DTOs instead of exposing entity models directly.
 - Enable structured logging.
 - Organize configuration using `appsettings.json` and environment-specific settings.
 - Write unit and integration tests.
 - Use Swagger/OpenAPI for documentation.
-

ASP.NET Core vs ASP.NET Framework

Feature	ASP.NET Framework	ASP.NET Core

Cross-Platform	No	Yes
Open Source	Limited	Yes
Performance	Good	Excellent
Dependency Injection	External Libraries	Built-in
Hosting	IIS Only	IIS, Kestrel, Nginx, Apache, Docker
Middleware Pipeline	Limited	Fully Modular
Cloud Support	Limited	Excellent
Minimal APIs	No	Yes
Microservices	Limited	Excellent
Future Development	Maintenance Mode	Active Development

A typical enterprise ASP.NET Core application follows this architecture:



Additional cross-cutting components include:

- Middleware
- Authentication
- Authorization
- Logging
- Caching
- Configuration
- Background Services
- Monitoring
- Health Checks

This layered architecture promotes maintainability, scalability, and testability, making it suitable for enterprise applications.

Summary

ASP.NET Core is Microsoft's flagship framework for building modern web applications and APIs. It combines high performance, cross-platform support, built-in Dependency Injection, a modular middleware pipeline, robust security features, and seamless

cloud integration into a unified development platform. Whether building RESTful APIs, MVC applications, Minimal APIs, microservices, or cloud-native solutions, ASP.NET Core provides the flexibility, scalability, and tooling required for enterprise-grade software development. Mastering ASP.NET Core is essential for any modern .NET developer, as it forms the foundation for building secure, maintainable, and high-performance applications in today's software ecosystem.